A Four-Level Inverse-Problem Case Study

MATLAB implementation for a synthetic microbial consortium

Technical implementation report accompanying the manuscript
A Hierarchical Framework for Inverse Problems in Biological Cybernetics

23 July 2026

Abstract

This report presents a reproducible MATLAB case study that turns the inverse ladder into a single biological example. We consider two microbial strains that compete for a supplied substrate while exchanging metabolites, and ask four progressively deeper questions: which kinetic parameters govern their dynamics, which growth law can be discovered from observations, whether their behavior can be interpreted as optimizing a community-level objective, and whether it is better explained as a strategic interaction between the two strains. Synthetic data with known ground truth make it possible to assess each level while keeping the biological system fixed.

The results show that parameter calibration and sparse model discovery can reconstruct the observed dynamics, while also exposing structural ambiguities that a good predictive fit alone cannot resolve. At the higher levels, inverse optimal control and inverse differential games identify objectives that reproduce, respectively, centralized and strain-specific allocation behavior within the simulated setting. The case study therefore illustrates the central message of the inverse ladder: moving upward changes the goal from estimating numerical coefficients to inferring mechanisms and behavioral explanations, but it also requires stronger assumptions and introduces new forms of non-uniqueness. The implementation is intended as a transparent proof of concept and as a basis for studying identifiability, experimental design, and robustness before applying these ideas to real microbial communities.

Contents

1 Purpose and scope

The accompanying manuscript [1] proposes a hierarchy in which each inverse problem inherits the uncertainties of the levels below it and introduces a qualitatively new source of non-uniqueness. A useful case study should therefore do more than present four unrelated algorithms. It should hold the biological setting fixed while changing what is assumed known and what is inferred.

The present implementation was designed around five requirements:

1. use the same microbial system at all four levels;
2. keep the dynamics small enough for transparent mathematical analysis;
3. introduce explicit, biologically interpretable control variables for Levels 3 and 4;
4. provide known ground truth through synthetic data; and
5. expose non-identifiability rather than forcing exact recovery.

The result is a proof-of-concept, not a claim that objectives can already be reliably inferred from ordinary microbial abundance data. In particular, dynamic allocation measurements or appropriate molecular proxies would be needed before applying the upper levels to a real consortium.

2 Biological system and dynamic model

2.1 Two-strain cross-feeding consortium

Two microbial strains X_1 and X_2 compete for a common substrate S. Strain 1 secretes metabolite M_1, which supports growth of strain 2, while strain 2 secretes metabolite M_2, which supports growth of strain 1. The state is

$$\boldsymbol{x}(t) = \begin{bmatrix} X_1(t) & X_2(t) & S(t) & M_1(t) & M_2(t) \end{bmatrix}^{\mathsf{T}}. \tag{1}$$

The dimensionless variable $u_i(t) \in [0, 1]$ represents the effective fraction of strain i's metabolic resources allocated to secretion. Increasing u_i increases secretion of M_i, but reduces the strain's instantaneous growth capacity. This construction creates a biologically interpretable trade-off between individual growth and support of the partner.

For Levels 1 and 2, the primary laboratory interpretation is an engineered consortium in which expression of each secretion pathway is actuated by an external input, such as an inducer concentration or an optogenetic light signal. Strictly speaking, the experimenter prescribes this actuator rather than directly setting an intracellular resource fraction. The model assumes that a previously calibrated actuator-to-allocation relation maps the applied signal to a known effective input $u_i(t)$. Thus, "prescribed allocation" is shorthand for a prescribed actuator with a known effective allocation; it does not imply perfect direct control of cellular metabolism.

A different experiment could estimate $u_i(t)$ from a transcriptional reporter, flux proxy, or other molecular measurement without externally actuating the pathway. In that case $u_i(t)$ would be a measured covariate, not an externally prescribed input, and its measurement error and possible dependence on the cellular state should be included in the inverse problem. That measured-only design is not simulated here.

Throughout the case study, time is measured in hours (h), biomass concentrations X_i in grams of dry weight per litre ($\mathrm{gDW\,L^{-1}}$), and extracellular substrate and metabolite concentrations S and M_i in millimoles per litre ($\mathrm{mmol\,L^{-1}}$). The allocation variables u_i are dimensionless.

The epistemic status of u_i changes deliberately along the inverse ladder. In Levels 1 and 2 it is a known exogenous input used to excite and identify the dynamics. In Level 3 it becomes an observed endogenous trajectory hypothesized to solve a centralized optimal-control problem. In Level 4, u_i is instead the endogenous strategy of strain i in a differential game. The four levels

therefore share the same biological plant and state equations, but the upper levels use separately generated control trajectories and ask a different question about their origin. Applying inverse optimal control or an inverse game directly to an externally imposed $u_i(t)$ could at most explain the experimenter's actuation policy; it would not identify a biological objective of the strains.

2.2 Growth and material-balance equations

For strain i, let $j = 3 - i$ denote its partner. The unburdened specific growth rate is

$$\bar{\mu}_i(\boldsymbol{x}) = \mu_{i,\max} \frac{S}{K_{S,i} + S} \frac{M_j}{K_{M,i} + M_j}, \tag{2}$$

and the allocation-dependent growth rate is

$$\mu_i(\boldsymbol{x}, u_i) = \bar{\mu}_i(\boldsymbol{x})(1 - c_i u_i). \tag{3}$$

The implementation enforces a small positive lower bound on the allocation factor for numerical robustness. The complete dynamics are

$$\dot{X}_i = (\mu_i(\boldsymbol{x}, u_i) - D)\, X_i, \tag{4}$$

$$\dot{S} = D(S_{\text{in}} - S) - \sum_{i=1}^{2} \frac{\mu_i(\boldsymbol{x}, u_i) X_i}{Y_{S,i}}, \tag{5}$$

$$\dot{M}_1 = \alpha_1 u_1 X_1 - \frac{\mu_2(\boldsymbol{x}, u_2) X_2}{Y_{M,2}} - D M_1, \tag{6}$$

$$\dot{M}_2 = \alpha_2 u_2 X_2 - \frac{\mu_1(\boldsymbol{x}, u_1) X_1}{Y_{M,1}} - D M_2. \tag{7}$$

Table 1 lists the ground-truth parameters used to generate the synthetic data.

Table 1: Ground-truth parameters of the synthetic microbial consortium.

Quantity	Strain 1	Strain 2	Unit
Maximum growth rate, $\mu_{i,\max}$	0.55	0.50	h^{-1}
Substrate half-saturation, $K_{S,i}$	0.80	1.00	$\mathrm{mmol\,L}^{-1}$
Metabolite half-saturation, $K_{M,i}$	0.12	0.10	$\mathrm{mmol\,L}^{-1}$
Allocation cost, c_i	0.35	0.30	dimensionless
Secretion rate, α_i	0.55	0.50	$\mathrm{mmol\,gDW}^{-1}\,\mathrm{h}^{-1}$
Substrate yield, $Y_{S,i}$	0.65	0.60	$\mathrm{gDW\,mmol}^{-1}$
Metabolite yield, $Y_{M,i}$	8.0	8.0	$\mathrm{gDW\,mmol}^{-1}$
Dilution rate, D	0.05		h^{-1}
Inlet substrate, S_{in}	10.0		$\mathrm{mmol\,L}^{-1}$

3 Synthetic experiments and numerical implementation

3.1 Information content, exogeneity, and identifiability

External actuation of u_i is a useful experimental-design choice for the lower levels, but it is neither a logical requirement nor a guarantee of identifiability. What is essential is that the observed trajectories contain enough independent information about the unknown quantities. For Level 1, local practical identifiability is related to the rank and conditioning of the stacked parameter-sensitivity matrix

$$\mathcal{S}_\theta = \begin{bmatrix} \partial \boldsymbol{x}(t_1)/\partial \boldsymbol{\theta} \\ \vdots \\ \partial \boldsymbol{x}(t_N)/\partial \boldsymbol{\theta} \end{bmatrix}. \tag{8}$$

If its columns are independent over the collection of experiments, parameter effects can in principle be separated locally; poor conditioning still makes their estimates sensitive to finite sampling and noise. For Level 2, the analogous requirement is that the columns of the regression library be sufficiently distinct over the sampled state–allocation region. In particular, the data should not make q_i, u_i, and $q_i u_i$ nearly collinear.

Rich endogenous allocations can satisfy these information conditions. Direct actuation of u_i is therefore not indispensable if its natural variation is measured accurately and the community visits a sufficiently broad set of states. However, endogeneity introduces an additional identification issue. If allocation is generated by feedback,

$$u_i(t) = \pi_i\big(\boldsymbol{x}(t)\big), \tag{9}$$

the observations explore the closed-loop trajectories produced jointly by the biological dynamics and the policy π_i. Different combinations of plant dynamics and feedback can then agree on the observed trajectories. Moreover, process disturbances or unmeasured physiological variables may correlate u_i with the equation error, so treating it as an ordinary error-free input can bias parameter estimates or favor an incorrect model term. Such data require a closed-loop identification treatment, a joint model of the allocation mechanism, suitable instrumental information, or an explicit measurement-error model.

External actuation helps by breaking some state–allocation correlations, expanding the explored operating region, and improving the conditioning of the sensitivity and library matrices. It cannot rescue an intrinsically non-identifiable model, and a constant or poorly designed actuator remains uninformative. Conversely, u_i may be left endogenous while initial conditions, inlet substrate S_{in}, dilution rate D, or other environmental conditions are varied across experiments. The resulting family of natural responses may also provide adequate excitation.

Thus, structural identifiability concerns uniqueness permitted by the model, observation scheme, and admissible inputs, whereas practical identifiability concerns the information and noise level in the realized dataset. The present case study uses exogenous allocation actuation to make the Level 1 and Level 2 demonstrations controlled and well conditioned, not because endogenous data are fundamentally unusable.

3.2 Calibration experiments

Three calibration experiments use different initial biomass ratios, substrate levels, and metabolite concentrations. For the synthetic engineered-consortium interpretation, the externally applied actuator signals are converted through the assumed calibrated input map into the effective allocations $u_1(t)$ and $u_2(t)$. These known inputs are persistently exciting square-wave schedules with small sinusoidal components and are imposed independently of the observed state trajectories. They span approximately 0.08 to 0.82. This variation is important because u_i affects both secretion through $\alpha_i u_i X_i$ and the growth burden through $c_i u_i$; varying it helps separate the secretion coefficients α_i from the allocation costs c_i. States are sampled at 49 times over 24 time units and corrupted with independent Gaussian noise at 0.5% of the maximum state scale.

3.3 Two simulation modes

The MATLAB implementation contains two complementary integrators that solve the same biological ODEs:

- `simulateCommunity.m` uses `ode15s` with interpolated controls and non-negativity constraints;
- `simulateControlledModel.m` uses fixed-step fourth-order Runge–Kutta integration for the repeated simulations required by optimization.

The adaptive `ode15s` solver is used as the reference integrator for generating or validating experimental trajectories. Its variable step size and error control are appropriate when accuracy and robustness to possible stiffness are more important than a fixed computational cost. It also supports continuous interpolation of measured control profiles and direct enforcement of state non-negativity.

The fixed-step Runge–Kutta solver is used inside the parameter-estimation, optimal-control, finite-difference, and best-response loops. These algorithms may evaluate the model thousands of times. A prescribed integration grid gives each evaluation the same number of right-hand-side calls, aligns exactly with the piecewise-constant control parameterization, and avoids small changes in adaptive step selection appearing as numerical irregularities in objective functions or finite-difference gradients. For this low-dimensional model, it is therefore substantially faster and more reproducible within optimization.

The two modes are not different models and are not expected to produce different biological conclusions. The fixed RK4 step was chosen only after a convergence check against `ode15s`; the unit tests require their all-state RMS difference to remain below 2×10^{-3} on a representative experiment. If the model were made substantially stiffer, this comparison would need to be repeated and the optimization integrator or step size revised.

3.4 Software entry point

The entire workflow is launched by

```
results = runInverseLadderCaseStudy;
```

The command generates data, executes the four levels, performs forward validation, saves `inverse_ladder_results.mat`, and exports the summary graphics.

4 Level 1: parameter estimation

4.1 Inverse problem

The model structure in equations (4)–(7) is assumed known. The effective allocation schedules $\boldsymbol{u}_e(t) = [u_{1,e}(t), u_{2,e}(t)]^\mathsf{T}$ are also known for each experiment e. Six quantities are estimated:

$$\boldsymbol{\theta} = [\mu_{1,\max}, \mu_{2,\max}, \alpha_1, \alpha_2, c_1, c_2]^\mathsf{T}. \tag{10}$$

All remaining parameters are treated as independently characterized. The weighted least-squares problem is

$$\hat{\boldsymbol{\theta}} = \arg \min_{\boldsymbol{\theta}_{\min} \le \boldsymbol{\theta} \le \boldsymbol{\theta}_{\max}} \sum_{e,k,\ell} \left[\frac{x^{\mathrm{sim}}_{e,k,\ell}(\boldsymbol{\theta}; \boldsymbol{u}_e) - x^{\mathrm{obs}}_{e,k,\ell}}{s_\ell} \right]^2, \tag{11}$$

where e, k, and ℓ index experiments, times, and states, respectively, and s_ℓ is a state-specific scale.

This is a parameter-estimation problem conditional on the applied inputs. Every candidate parameter vector is simulated under exactly the same $\boldsymbol{u}_e(t)$. The least-squares optimizer neither estimates these schedules nor changes them to improve the fit, and Level 1 makes no claim that they are optimal choices made by the microorganisms. This distinction is what is meant by "externally prescribed" in Figure 1.

The problem is solved with `lsqnonlin` from two dispersed initial points. Bound constraints enforce biologically meaningful values.

Compact MATLAB pseudocode

```
Input: experiments {t_e, xobs_e, u_e}, parameter bounds, initial guesses
for each initial guess theta0
    theta <- lsqnonlin(residual, theta0, bounds)
    residual(theta):
        for each experiment e
            xsim_e <- simulateControlledModel(theta, x0_e, t_e, u_e)
            append scaled(xsim_e - xobs_e)
select theta_hat with the smallest residual norm
Output: theta_hat and fitted state trajectories
```

4.2 Results

Table 2: Level 1 parameter-estimation results.

Parameter	True	Estimated	Relative error (%)
$\mu_{1,\max}$	0.55000	0.55080	0.145
$\mu_{2,\max}$	0.50000	0.49919	-0.162
α_1	0.55000	0.54964	-0.065
α_2	0.50000	0.50039	0.078
c_1	0.35000	0.35175	0.501
c_2	0.30000	0.29614	-1.287

The RMS relative error across all six parameters is 0.572%. Figure 1 shows the observed and fitted trajectories for the first calibration experiment. The fitted model reproduces biomass, substrate, and metabolite dynamics while being driven by the prescribed effective allocation schedules.

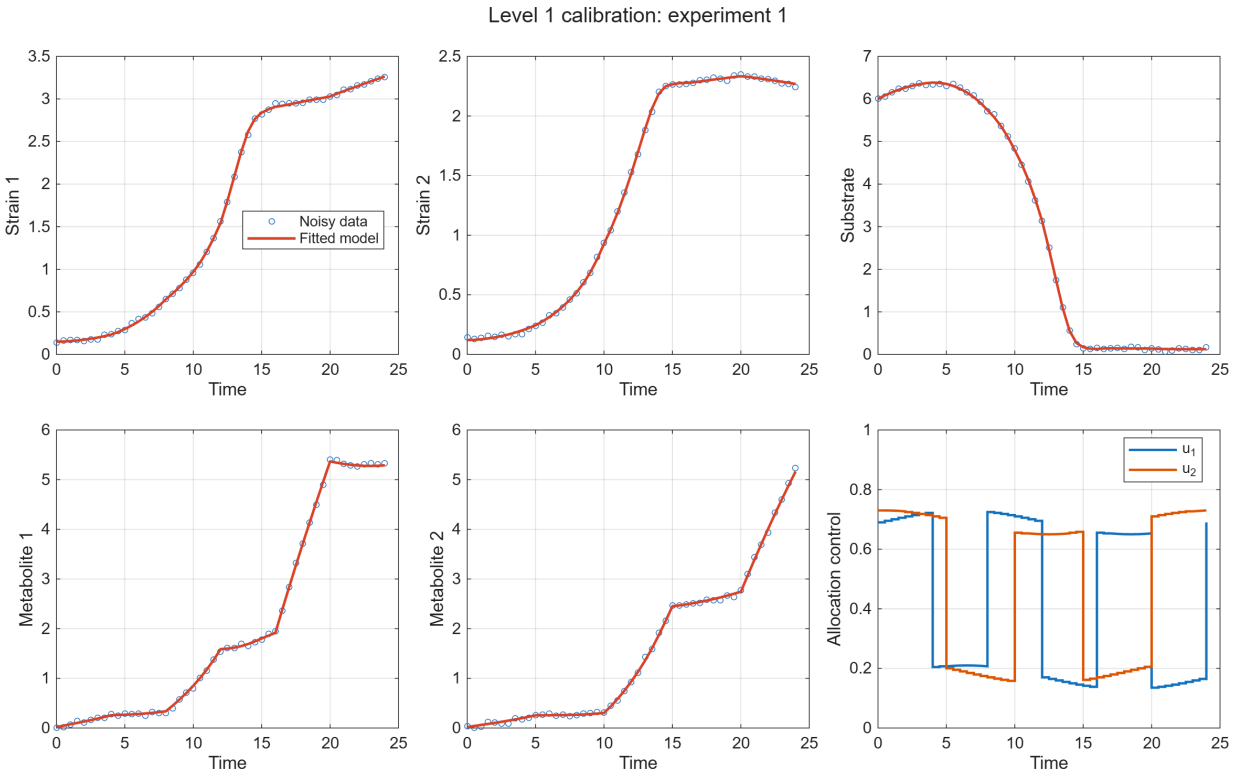

Figure 1: Level 1 calibration for the first synthetic experiment. Circles are noisy observations; solid curves are predictions obtained with the estimated parameters. The bottom-right panel shows the externally prescribed allocation inputs $u_i(t)$, interpreted as effective allocations produced by calibrated external actuation of the secretion pathways.

5 Level 2: model discovery

5.1 Candidate library

The environmental balances are retained, while the per-capita biomass growth law is treated as unknown. As in Level 1, the allocation schedules are known exogenous inputs rather than quantities discovered by the regression. This library-based sparse-regression construction follows the general data-driven model-discovery paradigm [2]. For each strain, the candidate library is constructed from:

$$\boldsymbol{\phi} = \begin{bmatrix} q_i & q_i u_i & X_1 & X_2 & S & M_j & u_i \end{bmatrix}, \qquad q_i = \frac{S}{K_{S,i} + S} \frac{M_j}{K_{M,i} + M_j}. \tag{12}$$

Rather than differentiating noisy biomass measurements directly, the method uses window-integrated per-capita growth:

$$\frac{\log X_i(t_b) - \log X_i(t_a)}{t_b - t_a} \approx \beta_{i,0} + \frac{1}{t_b - t_a} \int_{t_a}^{t_b} \boldsymbol{\phi}(t)^{\mathsf{T}} \boldsymbol{\beta}_i \, dt. \tag{13}$$

Savitzky–Golay smoothing is applied before integration. LASSO selects a sparse coefficient vector using cross-validation.

This construction is closely related to weak SINDy [3]. With $z_i = \log X_i$, the window equation is the integral, or constant-test-function, form of $\dot{z}_i = \beta_{i,0} + \boldsymbol{\phi}^{\mathsf{T}} \boldsymbol{\beta}_i$: it replaces a pointwise derivative by endpoint differences and integrals of the candidate library. General WSINDy instead uses a bank of smooth, compactly supported test functions and transfers derivatives to those functions by integration by parts. The present algorithm is therefore best viewed as a restricted weak-form

SINDy method using boxcar windows, explicit smoothing, cross-validated LASSO, and post-fit thresholding.

Compact MATLAB pseudocode

```
Input: calibration experiments and Level 1 parameter estimates
for each strain i
    smooth the measured state trajectories
    for each admissible time window
        compute window-averaged per-capita growth y
        compute averaged library row [q_i, q_i*u_i, X1, X2, S, Mj, u_i]
    betaPath, CV <- lasso(Phi, y)
    beta_i <- coefficients at minimum cross-validation error
    threshold small coefficients and compute R2
Output: sparse support and coefficients for both growth laws
```

5.2 Results and structural ambiguity

Table 3: Level 2 sparse growth-law coefficients. Zeros denote terms removed by regularization.

Candidate term	Strain 1	Strain 2
Intercept	-0.0529	-0.0252
Monod cross-feeding, q_i	0.5200	0.4157
Allocation interaction, $q_i u_i$	-0.2090	0
Strain 1 abundance	0	0
Strain 2 abundance	0	0
Substrate	0	0
Received metabolite	0	0
Direct allocation, u_i	0	-0.0496
R^2	0.9497	0.9458

For strain 1, the discovered support agrees with the generating model: growth depends on q_1 and the interaction $q_1 u_1$. For strain 2, the method selects a direct allocation term instead of $q_2 u_2$, despite achieving $R^2 = 0.946$. This is not treated as a software failure. It is an example of structural indistinguishability caused by correlated candidate functions over the observed region of state-control space. A high predictive score does not guarantee recovery of the correct biological mechanism.

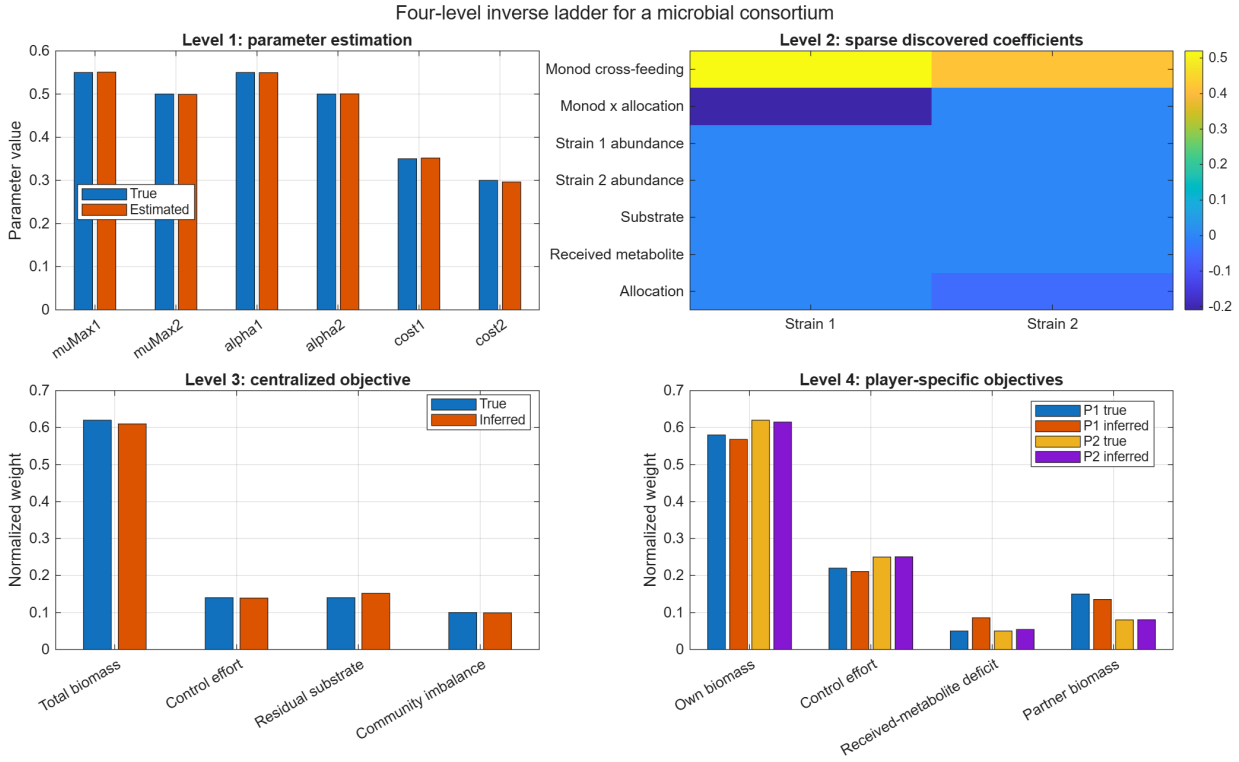

Figure 2: Summary of the four inverse levels. The Level 2 heat map displays the selected sparse coefficients, while the other panels compare ground-truth and estimated parameters or objective weights.

6 Level 3: inverse optimal control

6.1 Centralized-planner hypothesis

In contrast to Levels 1 and 2, the allocation trajectories are now endogenous: they are observations to be explained rather than actuated identification inputs. At Level 3, the consortium is provisionally treated as a single decision maker choosing both allocation trajectories. This is a hypothesis to be tested, not an assertion of community-level selection. Multilevel optimization has previously been used for forward metabolic modeling of microbial communities [6]; here the objective is instead inferred from dynamic trajectories. The finite-horizon objective is linear in four normalized kernels:

$$J_C(\boldsymbol{u};\boldsymbol{w}) = \sum_{m=1}^{4} w_m F_m(\boldsymbol{x},\boldsymbol{u}), \qquad \boldsymbol{w} \geq 0, \qquad \mathbf{1}^{\mathsf{T}}\boldsymbol{w} = 1. \tag{14}$$

The kernels represent:

1. negative total biomass, so minimization favors biomass production;
2. quadratic allocation effort;
3. residual substrate;
4. squared biomass imbalance $(X_1 - X_2)^2$.

The forward optimal-control problem uses piecewise-constant controls on eight intervals and fourth-order Runge–Kutta integration. It is solved with `fmincon` and box constraints $0.02 \leq u_i \leq 0.95$.

6.2 Relation to bilevel inverse optimal control

The general inverse optimal-control framework of Tsiantis, Balsa-Canto, and Banga [4] was formulated as a bilevel problem. In a simplified version with observed allocation controls and known dynamics, the structure would be

$$\hat{\boldsymbol{w}} = \arg\min_{\boldsymbol{w}\in\Delta} D\Big(\boldsymbol{u}^{\mathrm{obs}}, \boldsymbol{u}^*(\boldsymbol{w})\Big), \qquad \boldsymbol{u}^*(\boldsymbol{w}) \in \arg\min_{\boldsymbol{u}\in\mathcal{U}} J_C(\boldsymbol{u};\boldsymbol{w}), \tag{15}$$

where $\Delta = \{\boldsymbol{w} \geq 0 : \mathbf{1}^{\mathsf{T}}\boldsymbol{w} = 1\}$, $\mathcal{U}$ includes the dynamic and control constraints, and D is a trajectory-mismatch metric. More general versions can estimate model parameters and unmeasured controls simultaneously. A direct numerical solution of (15) would call the forward optimal-control solver inside every outer evaluation of a candidate $\boldsymbol{w}$.

Because that nested problem is demanding, the practical strategy in [4] decomposed it into input and parameter reconstruction, generation of a multiobjective Pareto set of optimal controls, and selection of the Pareto solution most compatible with the data. The MATLAB case study implemented here uses neither a direct bilevel solve nor that Pareto-set decomposition. It assumes that the kinetic model has already been calibrated and that the endogenous allocation trajectories are observed. It then replaces the inner optimization problem by first-order stationarity conditions at those observed trajectories.

6.3 Inverse stationarity formulation

Because the objective is linear in $\boldsymbol{w}$, its control gradient at an observed optimum $\boldsymbol{u}^*$ can be written

$$\nabla_{\boldsymbol{u}} J_C(\boldsymbol{u}^*;\boldsymbol{w}) = \begin{bmatrix}\nabla_{\boldsymbol{u}} F_1 & \cdots & \nabla_{\boldsymbol{u}} F_4\end{bmatrix}\boldsymbol{w} = A_C\boldsymbol{w}. \tag{16}$$

Finite differences provide the feature gradients. Rows associated with active control bounds are excluded when enough interior controls are available. The inverse problem is

$$\hat{\boldsymbol{w}} = \arg\min_{\boldsymbol{w}\geq 0,\ \mathbf{1}^{\mathsf{T}}\boldsymbol{w}=1} \|A_C\boldsymbol{w}\|_2^2 + \lambda\|\boldsymbol{w}-\boldsymbol{w}_0\|_2^2, \tag{17}$$

with a very small regularization parameter $\lambda = 10^{-8}$ and uniform reference $\boldsymbol{w}_0$.

The singular values of A_C provide a diagnostic of how much information the observed controls contain about the objective weights. If $A_C = U\Sigma V^{\mathsf{T}}$, each right singular vector in V represents an orthogonal combination of objective weights, and its corresponding singular value measures how strongly that combination affects the stationarity residual. A large singular value denotes a weight direction that is readily distinguished because it produces a large violation of stationarity. A near-zero singular value instead identifies a combination of weights that is compatible with the observed optimum. Because multiplying an objective by a positive constant does not change its optimizer, the simplex constraint $\mathbf{1}^{\mathsf{T}}\boldsymbol{w} = 1$ removes this unavoidable scale ambiguity. After normalization, one isolated small singular value supports a locally well-defined objective direction, whereas several comparably small singular values would indicate that multiple objectives are practically indistinguishable from the available demonstrations. In numerical data, an exact zero is generally replaced by a small value because of finite-difference error, discretization, and noise.

Compact MATLAB pseudocode

```
Input: model, initial states, candidate community features
for each synthetic experiment e
    demo_e <- solveCommunityPlanner(model, x0_e, trueWeights)
    A_e <- finiteDifferenceFeatureJacobian(demo_e, communityCostFeatures)
    retain stationarity rows for interior controls and stack into A_C
w_hat <- inferSimplexWeights(A_C, lambda)
for each experiment e
    validation_e <- solveCommunityPlanner(model, x0_e, w_hat,
                                          initial controls = 0.35)
Output: inferred weights and control/state validation errors
```

Thus, `solveCommunityPlanner` is used to create the synthetic demonstrations and, after inference, to perform forward validation. It is not nested inside `inferSimplexWeights`; the inverse calculation itself is a constrained linear least-squares problem solved with `lsqlin`. This is substantially cheaper than bilevel optimization, but it establishes only compatibility with local first-order conditions. Stationarity alone does not exclude a saddle point, a local maximum, or a non-global local minimum, and it does not jointly reconstruct unknown controls or kinetic parameters. Forward re-optimization is therefore an essential validation step, but does not turn the inverse calculation into a bilevel method. To make this check independent of the demonstrated solution, every validation solve is cold-started from the uniform default $u_1 = u_2 = 0.35$ on all control intervals. The observed controls are deliberately not supplied as the initial guess: doing so would start `fmincon` essentially at the answer and could produce zero trajectory error after a single iteration, which is only a warm-start consistency check.

6.4 Results

Table 4: Level 3 centralized objective weights.

Objective kernel	True	Inferred
Total biomass	0.6200	0.6098
Control effort	0.1400	0.1391
Residual substrate	0.1400	0.1519
Community imbalance	0.1000	0.0991

The Euclidean weight error is 1.57×10^{-2}. Both cold-started forward solves converged with a positive `fmincon` exit flag and closely reproduced the demonstrations, but not identically; Table 5 reports the resulting nonzero discrepancies.

Table 5: Level 3 cold-start forward validation. RMSE values compare the re-optimized trajectories obtained with the inferred weights against the synthetic demonstrations.

Experiment	Iterations	Control RMSE	State RMSE
1	46	4.491×10^{-5}	5.182×10^{-5}
2	44	1.326×10^{-5}	1.955×10^{-5}

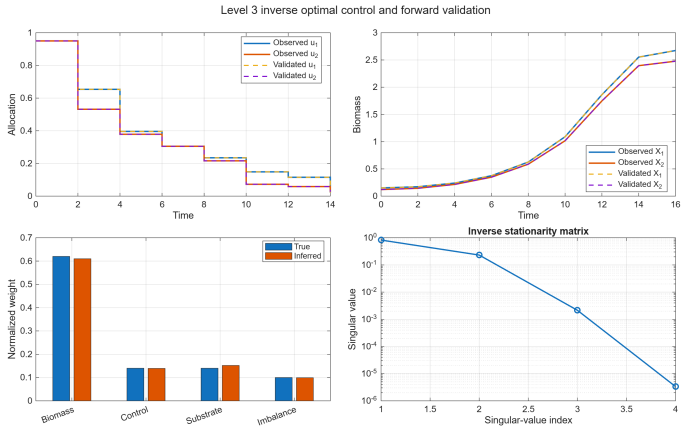

Figure 3: Level 3 results for the first experiment. Starting from the uniform default allocation $u_1 = u_2 = 0.35$, the forward solve with the inferred objective closely reproduces the observed optimal controls and biomass trajectories. The bottom-right panel shows the singular-value spectrum of the inverse stationarity matrix.

The singular values of the stacked stationarity matrix are approximately

$$8.23 \times 10^{-1}, \quad 2.31 \times 10^{-1}, \quad 2.2 \times 10^{-3}, \quad \text{near zero.}$$

Here the final, near-zero singular value corresponds to the objective-weight direction that best satisfies the observed stationarity conditions. The next smallest singular value represents the closest competing direction. Their ratio is approximately 641, so the preferred direction is well separated from that alternative and the normalized objective is locally well defined for these synthetic demonstrations. This is a local identifiability diagnostic, not proof that the same objective would remain identifiable under a different feature library, noisier measurements, or less informative trajectories.

7 Level 4: inverse differential game

7.1 Player-specific objectives

The centralized hypothesis is replaced by two independent decision makers, following the inverse noncooperative dynamic-game perspective [5]. Here each $u_i(t)$ is interpreted as strain i's endogenous strategy, rather than as an externally actuated input. Player i minimizes

$$J_i(u_i, u_j; \boldsymbol{w}_i) = \sum_{m=1}^{4} w_{i,m} F_{i,m}, \qquad \boldsymbol{w}_i \geq 0, \qquad \mathbf{1}^{\mathsf{T}} \boldsymbol{w}_i = 1. \tag{18}$$

The four kernels are:

1. negative own biomass;
2. own quadratic allocation effort;
3. deficit of the metabolite received from the partner;
4. negative partner biomass.

The final term permits a general-sum interaction: supporting the partner can be beneficial because the partner supplies an essential metabolite.

An open-loop Nash equilibrium (u_1^*, u_2^*) satisfies

$$J_i(u_i^*, u_j^*) \leq J_i(u_i, u_j^*) \quad \text{for every feasible unilateral deviation } u_i. \tag{19}$$

7.2 Forward and inverse algorithms

The forward game is solved by damped iterative best response:

1. fix u_2 and optimize u_1;
2. fix the updated u_1 and optimize u_2;
3. damp the updates and repeat;
4. verify that neither player obtains a material unilateral improvement.

Each best response is an `fmincon` problem. The inverse problem is formed separately for each player by differentiating its four feature functionals with respect to its own control:

$$\hat{\boldsymbol{w}}_i = \arg \min_{\boldsymbol{w}_i \geq 0,\ \mathbf{1}^{\mathsf{T}} \boldsymbol{w}_i = 1} \|A_i \boldsymbol{w}_i\|_2^2 + \lambda \|\boldsymbol{w}_i - \boldsymbol{w}_0\|_2^2. \tag{20}$$

Compact MATLAB pseudocode

```
Input: model, initial states, candidate features for each player
for each synthetic experiment e
    demo_e <- solveOpenLoopNash(model, x0_e, truePlayerWeights)
    for each player i
        A_i,e <- finite-difference own features with respect to own control
        retain interior-control rows and stack into A_i
for each player i
    w_hat_i <- inferSimplexWeights(A_i, lambda)
validation <- solveOpenLoopNash(model, initial states, w_hat)
verify control/state errors and unilateral-improvement gaps
Output: player-specific weights and Nash validation diagnostics
```

7.3 Results

Table 6: Level 4 player-specific objective weights.

	Player 1		Player 2	
Objective kernel	True	Inferred	True	Inferred
Own biomass	0.5800	0.5679	0.6200	0.6148
Control effort	0.2200	0.2110	0.2500	0.2506
Metabolite deficit	0.0500	0.0856	0.0500	0.0541
Partner biomass	0.1500	0.1355	0.0800	0.0804

The Euclidean weight errors are 4.13×10^{-2} and 6.68×10^{-3} for Players 1 and 2. Forward validation gives:

- mean control RMS error 1.02×10^{-3};
- mean state RMS error 9.32×10^{-4};
- maximum post-solution unilateral improvement below 7.4×10^{-6}.

The near-null-direction separation ratios are approximately 20.2 and 64.0 for the two players. Player 1 is less well-conditioned, consistent with its larger weight error, particularly for the received-metabolite term.

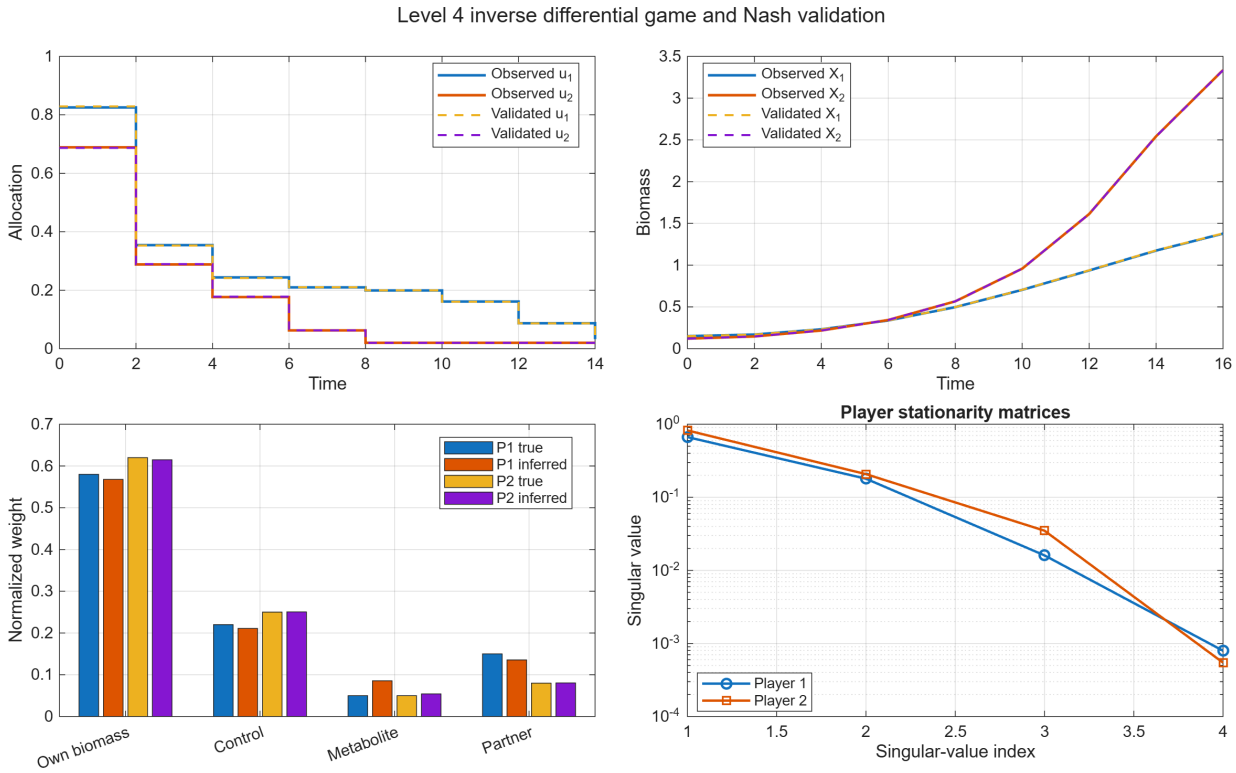

Figure 4: Level 4 results for the first experiment. Dashed forward-validation controls and trajectories closely track the equilibrium data. The lower panels compare true and inferred objectives and show the two stationarity spectra.

8 Implementation map and verification

Table 7: Principal MATLAB components.

File	Purpose
`runInverseLadderCaseStudy.m`	Top-level reproducible workflow.
`defaultInverseLadderConfig.m`	Parameters, experiments, objectives, and solver settings.
`communityRhs.m`	Five-state controlled microbial ODE.
`simulateCommunity.m`	Adaptive `ode15s` simulator.
`simulateControlledModel.m`	Fixed-step RK4 simulator used in optimization.
`runLevel1ParameterEstimation.m`	Bound-constrained multi-experiment calibration.
`runLevel2ModelDiscovery.m`	Integrated sparse growth-law discovery.
`solveCommunityPlanner.m`	Centralized forward optimal-control solver.
`runLevel3InverseOptimalControl.m`	Level 3 inverse stationarity and validation.
`solveOpenLoopNash.m`	Damped iterative best-response solver.
`runLevel4InverseDifferentialGame.m`	Player-wise inverse stationarity and Nash validation.
`testInverseLadderCaseStudy.m`	MATLAB unit tests.

The verified development environment is MATLAB R2026a Update 1 with Optimization Toolbox, Global Optimization Toolbox, Statistics and Machine Learning Toolbox, Signal Processing Toolbox, Symbolic Math Toolbox, Parallel Computing Toolbox, and Control System Toolbox. The current implementation uses the optimization, statistics, and signal-processing functionality directly.

Table 8: Verification summary.

Check	Result
Complete workflow runtime without plotting	approximately 14 s
MATLAB unit tests	4 of 4 passed
MATLAB Code Analyzer findings	0
Saved numerical result file	verified
Saved summary figure	verified
Nash unilateral-improvement check	$< 7.4 \times 10^{-6}$

9 Interpretation and limitations

9.1 What the case study demonstrates

The case study provides a computationally coherent realization of the inverse ladder:

- Level 1 recovers continuous kinetic parameters when the structure and controls are known.
- Level 2 admits competing structural explanations even when prediction is accurate.
- Level 3 recovers an objective only after choosing candidate kernels, normalizing their scale, and assuming centralized optimality.
- Level 4 requires an equilibrium concept, player-specific actions, and separate stationarity conditions.

The increasing data and modeling requirements are visible in the code. Levels 1 and 2 use ordinary time-series observations and prescribed inputs. Levels 3 and 4 additionally require controls, a candidate objective basis, and a forward optimal-control or game solver.

9.2 Important limitations

1. **Synthetic ground truth.** The trajectories, objectives, and controls are generated by the same model class used for inference. This is appropriate for verification but does not establish biological validity.
2. **Observed controls.** Real microbial studies rarely measure metabolic allocations directly. Transcript, protein, or secretion measurements would be required as proxies.
3. **Objective basis dependence.** Objective recovery is conditional on the selected feature library. An omitted kernel can redistribute inferred weights without visibly degrading the fit.
4. **Open-loop equilibrium.** Level 4 infers finite-horizon open-loop strategies. General nonlinear feedback Nash equilibria would require solving coupled Hamilton–Jacobi equations and are not attempted.
5. **Two-player scale.** The best-response and finite-difference methods are suitable for a compact illustration, not yet for large microbial communities.
6. **Local optimality information.** Inverse stationarity establishes compatibility with first-order conditions. Forward re-optimization and unilateral-deviation tests are therefore essential.

10 Recommended extensions

The most informative next developments are:

1. design additional asymmetric perturbations specifically to resolve the Level 2 allocation ambiguity;
2. add measurement noise to the Level 3 and Level 4 controls and quantify confidence regions for the inferred weights;
3. compare the centralized and game formulations on the same behavior to demonstrate cooperative mimicry explicitly;
4. replace synthetic allocations with gene-expression or metabolite-flux measurements from an engineered two-strain consortium;
5. add profile-based or Bayesian uncertainty quantification for objective weights;
6. test whether inferred objectives predict behavior under dilution, nutrient, or partner-knockdown conditions excluded from inference.

11 Conclusions

The MATLAB implementation shows that all four levels can be represented in a single biological and computational framework. Parameter estimation and sparse model discovery are inexpensive and mature enough for routine application. Inverse optimal control is feasible for a small controlled community when allocation trajectories are available. A two-player inverse differential game is also computationally feasible under open-loop, finite-horizon assumptions, but it introduces a substantially larger modeling commitment and more severe conditioning issues.

The central lesson is not that the upper levels produce uniquely correct biological objectives. Rather, the implementation demonstrates how to formulate those hypotheses, diagnose their identifiability, and test their predictive consequences. The Level 2 structural ambiguity and the Level 4 conditioning difference between players are therefore substantive results: they make the case study consistent with the manuscript's emphasis on both opportunities and limitations.

References

[1] J. R. Banga and A. F. Villaverde, *A Hierarchical Framework for Inverse Problems in Biological Cybernetics: Opportunities and Limitations*, manuscript, 2026.

[2] S. L. Brunton and J. N. Kutz, *Data-Driven Science and Engineering: Machine Learning, Dynamical Systems, and Control*, Cambridge University Press, 2022.

[3] D. A. Messenger and D. M. Bortz, "Weak SINDy: Galerkin-based data-driven model selection," *Multiscale Modeling & Simulation*, vol. 19, no. 3, pp. 1474–1497, 2021. doi: 10.1137/20M1343166.

[4] N. Tsiantis, E. Balsa-Canto, and J. R. Banga, "Optimality and identification of dynamic models in systems biology: an inverse optimal control framework," *Bioinformatics*, vol. 34, no. 14, pp. 2433–2440, 2018. doi: 10.1093/bioinformatics/bty139.

[5] T. L. Molloy, J. Inga, S. Hohmann, and T. Perez, *Inverse Optimal Control and Inverse Noncooperative Dynamic Game Theory*, Springer, 2022.

[6] A. R. Zomorrodi and C. D. Maranas, "OptCom: A multi-level optimization framework for the metabolic modeling and analysis of microbial communities," *PLoS Computational Biology*, vol. 8, no. 2, e1002363, 2012.